

# KR k-Length Computing: A Variable-Length Native Arithmetic Architecture for Computational Number Theory\*

H. Ismail  
Kyudoka Research  
ismh@kyudoka.org

## Abstract

Multi-precision integer arithmetic, as required by computational number theory and cryptographic workloads, is conventionally implemented either in software libraries (GMP, Pari/GP) running on fixed-width processors or in dedicated hardware accelerators whose operand widths are frozen at design time. Both approaches impose a fundamental mismatch: the former serialises inherently parallel word-level operations, while the latter cannot adapt to operand lengths that vary across algorithmic phases. We introduce *k-length computing*, an architecture in which the operand length  $k$  (measured in machine words) is a first-class runtime parameter. A hardware decomposition engine transforms variable-length multi-precision instructions into streams of word-level micro-operations whose carry-chain dependencies are encoded as dataflow graph edges. These micro-operations are dispatched to heterogeneous functional units—FPGA DSP slices, GPU integer cores, or networked compute nodes—via the KR FPGA Scheduler, an out-of-order scheduler derived from Tomasulo’s algorithm and extended with a carry-aware common data bus. We describe the three-level instruction set (L0 word-level, L1 multi-precision, L2 algorithmic), its encoding as RISC-V custom instructions on the NEORV32 soft processor, the decomposition engine’s micro-op generation for schoolbook and Karatsuba multiplication, a speculative carry prediction mechanism, and a passive machine-learning observation tap for learned scheduling policies. We present FPGA resource estimates for Xilinx 7-Series targets and analyse performance projections for 1024-bit and 4096-bit modular multiplication, with applications to the elliptic curve method (ECM), the quadratic sieve (MPQS), and the general number field sieve (GNFS).

**Keywords:** multi-precision arithmetic, RISC-V, FPGA, out-of-order scheduling, computational number theory, elliptic curve method, k-length computing

## 1 Introduction

The arithmetic engines at the core of computational number theory—integer factorisation, primality proving, discrete logarithm computation—operate on integers whose bit-lengths range from hundreds to tens of thousands of bits, varying not only between problems but within a single algorithmic run. The elliptic curve method for factorisation [7], for instance, performs modular arithmetic on integers of bit-length equal to the factor being sought (typically 20–70 digits), while the sieving phase of the general number field sieve [10] requires operations on considerably larger algebraic norms. In every case, the multi-precision arithmetic must decompose into word-sized primitives—add-with-carry, multiply-accumulate, double-width-divide—whose sequencing is dictated by data-dependent carry chains.

Contemporary approaches to this decomposition fall into two categories, neither satisfactory.

---

\*KR k-Length Computing is a product architecture developed by Kyudoka Research.

**Software multi-precision libraries.** GMP [6], Pari/GP [17], and similar libraries implement multi-precision arithmetic as loops of word-level instructions executing on fixed-width general-purpose processors. The inner loops are hand-tuned in assembly for each target microarchitecture. On architectures that provide a carry flag (x86, ARM), the `adc/sbb` instructions chain naturally. On RISC-V, which deliberately omits condition flags [16], carry propagation requires an additional `sltu` instruction after each addition, inflating the critical path by 40–100% in the inner loops of multi-precision addition and multiply-accumulate (Section 2.2).

**Fixed-width hardware accelerators.** Cryptographic accelerators for RSA, elliptic curve cryptography (ECC), and lattice-based schemes [4, 19] implement Montgomery multiplication [12] or NTT butterflies at specific operand widths (e.g., 256-bit for Ed25519, 2048-bit for RSA). These designs deliver high throughput at their design point but cannot serve a number-theoretic workload in which the same hardware must multiply 192-bit, 384-bit, and 1024-bit integers in rapid succession during, say, a multi-stage cofactorisation pipeline [11].

**Contribution.** We propose *KR  $k$ -Length Computing*, an architecture in which the operand length  $k$  (in 32-bit words) is a runtime parameter, not an architectural constant. The design has three distinguishing features:

1. A *decomposition engine* that accepts multi-precision instructions with operand-length parameters and emits word-level micro-operations tagged with their carry-chain dependencies, forming a dataflow graph suitable for out-of-order execution (Section 4).
2. The *KR FPGA Scheduler* (model KR-4804), an out-of-order micro-operation scheduler based on Tomasulo’s algorithm [18] and extended with a carry-aware common data bus (CDB) and a 10-bit tag space supporting up to 1,024 in-flight operations (Section 5).
3. A three-level instruction set encoded in the RISC-V custom-0 opcode space, implementable as a Custom Functions Unit (CFU) on the NEORV32 soft processor [13], providing software compatibility with Pari/GP’s multi-precision kernel primitives (Section 3).

The architecture targets heterogeneous compute fabrics: a single KR FPGA Scheduler instance may dispatch micro-operations to FPGA DSP48E1 slices [20], GPU integer pipelines, or remote compute nodes connected via a network CDB protocol. This heterogeneity is particularly relevant to the large-scale factorisations (ECM, MPQS, GNFS) central to computational number theory, where the inner multi-precision arithmetic benefits from hardware acceleration while the outer algorithmic structure distributes across machines.

The remainder of this paper is organised as follows. Section 2 reviews multi-precision arithmetic primitives, the RISC-V carry flag problem, and existing accelerator approaches. Section 3 defines the  $k$ -length instruction set at three levels of abstraction. Section 4 describes the decomposition engine. Section 5 presents the KR FPGA Scheduler’s carry-aware extensions. Section 6 analyses applications to number-theoretic algorithms. Section 7 introduces the ML-observable scheduling tap. Section 8 provides FPGA implementation details and resource estimates. Section 9 discusses limitations and related work. Section 10 concludes.

## 2 Background

### 2.1 Multi-Precision Arithmetic Primitives

The inner kernels of multi-precision arithmetic libraries reduce to a small set of word-level primitives. We adopt the naming conventions of the Pari/GP kernel [3, 17], which provides a convenient reference model for our hardware instruction set.

**Definition 1** (Word-level arithmetic primitives). Let  $W = 2^w$  be the machine word size (we take  $w = 32$  throughout). The following primitives operate on  $w$ -bit unsigned integers, with a hi-remainder shadow register `hireg`  $\in [0, W)$  and a carry flag  $c \in \{0, 1\}$ :

- (i) `addwc(a, b)`: Computes  $r = a + b + c$ , where  $r \bmod W$  is returned and  $c \leftarrow \lfloor (a + b + c)/W \rfloor$ .
- (ii) `subwb(a, b)`: Computes  $r = a - b - c$ , returning  $r \bmod W$  and updating  $c$  to the borrow.
- (iii) `mulfull(a, b)`: Computes the full  $2w$ -bit product  $a \cdot b$ . The low  $w$  bits are returned; the high  $w$  bits are stored in `hireg`.
- (iv) `addmul(a, b)`: Computes  $a \cdot b + \text{hireg}$ . The low  $w$  bits are returned; the high  $w$  bits update `hireg`. This is the multiply-accumulate primitive at the core of multi-precision multiplication.
- (v) `divdw(a, d)`: Computes  $q = \lfloor (\text{hireg} \cdot W + a)/d \rfloor$ , returning  $q$  and setting `hireg`  $\leftarrow (\text{hireg} \cdot W + a) \bmod d$ .
- (vi) `clz(a)`: Returns the number of leading zero bits in  $a$  (returns  $w$  if  $a = 0$ ).

These primitives are sufficient to implement the standard multi-precision algorithms: school-book multiplication ( $O(n^2)$  `addmul` operations for  $n$ -word operands), Karatsuba multiplication [8] ( $O(n^{\log_2 3})$ ), Montgomery reduction [12], and Barrett reduction. The algorithms in [2, 9] are expressed in terms of these primitives.

In the Pari/GP kernel, these primitives appear as the macros `addll`, `addllx`, `mulll`, `addmul`, `divll`, and `bfffo`, with platform-specific assembly implementations for each supported architecture. The global variables `hiremainder` and `overflow` serve as the shadow register and carry flag, respectively.

## 2.2 The RISC-V Carry Flag Problem

The RISC-V ISA [16] deliberately omits condition flags, including the carry and overflow flags that are fundamental to efficient multi-precision arithmetic on x86 (`adc`, `sbc`) and ARM (`adcs`, `sbc`s). The ISA specification justifies this decision on the grounds of simplifying out-of-order microarchitectures by eliminating implicit register dependencies. The consequence for multi-precision arithmetic is severe.

Consider the Pari/GP `addll` macro for RISC-V (from `kernel/riscv64/asm0.h`):

```
1 add %0, %2, %3      # result = a + b
2 sltu %1, %4, %2      # overflow = (result < a) ? 1 : 0
```

Listing 1: Pari/GP `addll` for RISC-V: two instructions where x86 requires one (`add+adc`).

The `addllx` (add with carry-in from a previous operation) expands to five instructions:

```
1 add %0, %3, %4      # tmp = a + b
2 sltu %2, %5, %3      # c1 = (tmp < a)
3 add %0, %5, %6      # result = tmp + carry_in
4 sltu %1, %5, %6      # c2 = (result < carry_in)
5 add %1, %7, %6      # overflow = c1 + c2
```

Listing 2: Pari/GP `addllx` for RISC-V: five instructions per carry-propagating addition.

Similarly, the `addmul` primitive—the innermost operation in multi-precision multiplication—requires five RISC-V instructions (`mulhu`, `mul`, `add`, `sltu`, `add`) where a single multiply-accumulate with carry would suffice on a machine with appropriate support.

Table 1 summarises the instruction-count reduction achieved by the KR k-Length Computing custom function unit (CFU) instructions relative to the standard RISC-V implementations in Pari/GP. The savings are most dramatic for `divll`, which requires approximately 40 instructions

Table 1: Instruction counts per invocation of Pari/GP kernel primitives on RISC-V (standard ISA) versus KR k-Length Computing CFU instructions. The RISC-V column counts only ALU instructions, excluding loads, stores, and control flow.

| Primitive | RISC-V (standard) | KR CFU | Speedup (instructions) |
|-----------|-------------------|--------|------------------------|
| addll     | 2                 | 1      | 2×                     |
| addllx    | 5                 | 1      | 5×                     |
| subll     | 2                 | 1      | 2×                     |
| subllx    | 5                 | 1      | 5×                     |
| mulll     | 2                 | 1      | 2×                     |
| addmul    | 5                 | 1      | 5×                     |
| divll     | ~40               | 1      | ~40×                   |
| bfffo     | ~12               | 1      | ~12×                   |

in the software implementation (a loop-based restoring division) and reduces to a single hardware instruction with 34-cycle latency.

The critical observation is not merely that each primitive becomes cheaper, but that the *carry flag is internalised* within the CFU. The shadow register `HIREG` and the carry bit `c` persist across consecutive custom instructions without occupying general-purpose registers or requiring explicit shuttling. This eliminates the serial dependency chain that RISC-V’s flag-free design imposes on multi-precision inner loops.

### 2.3 Fixed-Width Accelerators

Hardware accelerators for public-key cryptography—RSA engines, ECC co-processors, and lattice-based cryptography accelerators—are designed around specific operand widths. An RSA accelerator for 2048-bit keys implements a 2048-bit Montgomery multiplier [12]; an ECC accelerator for the NIST P-256 curve implements 256-bit modular arithmetic. The datapath width is a design-time constant wired into the silicon or FPGA configuration.

This rigidity is acceptable for TLS termination or secure boot, where the key size is known in advance. It is entirely unsuitable for computational number theory, where the working precision varies:

- In ECM [7], Stage 1 and Stage 2 operate modulo the number being factored, whose size is the algorithm’s input. Different factorisation campaigns target different factor sizes, requiring different working precisions.
- In MPQS [15], the factor base bound and sieving range determine the size of intermediate values. The linear algebra phase operates over  $\text{GF}(2)$  matrices whose dimensions depend on the factor base.
- In GNFS [10], the polynomial selection, sieving, and linear algebra phases each demand different operand sizes. The matrix step requires arithmetic modulo large primes, while sieving uses comparatively small values.

A fixed-width accelerator must either be over-provisioned (wasting area and power on the common case) or under-provisioned (falling back to software for large operands). Neither option is acceptable when the goal is to maximise throughput across a range of operand sizes within a single computation.

## 2.4 GPU Multi-Precision Arithmetic

GPUs have been applied to multi-precision arithmetic for cryptanalysis [5, 11, 21, 22] with mixed results. The fundamental challenge is that GPU warps execute in SIMT fashion with 32-bit integer ALUs, and multi-precision carry propagation is inherently sequential within a single multi-word addition.

Existing GPU multi-precision libraries (cuBignum, CGBN) decompose large integers across warp lanes, using warp-shuffle instructions to propagate carries. For a  $k$ -word integer distributed across 32 lanes, the carry propagation requires  $\lceil k/32 \rceil$  shuffle rounds, each adding latency. When  $k \leq 32$  (i.e., operands fit in a single warp), the approach works reasonably. For  $k > 32$ , multiple warps must coordinate via shared memory, and the overhead becomes substantial.

GPU-ECM implementations [22] have demonstrated that the *outer* parallelism of ECM (running many independent curves simultaneously) maps well to GPU architectures, even when the *inner* multi-precision arithmetic is not individually fast. This observation motivates our heterogeneous scheduling approach (Section 5): the KR FPGA Scheduler can dispatch the inner word-level operations to FPGA DSP slices for latency-optimal execution while an outer scheduler distributes independent curve instances across GPU warps or networked nodes.

## 2.5 The KR FPGA Scheduler: Foundations

The KR FPGA Scheduler (KR-4804) is an out-of-order instruction scheduler designed for FPGA implementation. Its theoretical foundation is Tomasulo’s algorithm [18], originally devised for the IBM System/360 Model 91 floating-point unit. The algorithm uses *reservation stations* to buffer instructions waiting for operands and a *common data bus* (CDB) to broadcast results to all waiting consumers, enabling out-of-order execution while preserving data dependencies.

In the original algorithm, each issued instruction receives a *tag* identifying the reservation station that will produce its result. Dependent instructions record this tag rather than the result value; when the producing instruction completes and broadcasts its result on the CDB, all consumers snoop the broadcast and capture the value.

The KR FPGA Scheduler extends this model in several ways relevant to  $k$ -length computing:

1. **Wider tag space.** The original Tomasulo implementation used 3-bit tags (8 reservation stations). The KR-4804 uses 10-bit tags, supporting up to 1,024 in-flight micro-operations—necessary when a single 1024-bit multiply decomposes into  $32 \times 32 = 1024$  word-level `addmul` operations.
2. **Carry-aware CDB.** Each CDB broadcast includes not only the  $w$ -bit result value but also a 1-bit carry flag, enabling dependent add-with-carry operations to receive their carry input through the same snooping mechanism used for data operands.
3. **Heterogeneous dispatch.** Reservation stations are partitioned by functional unit type (FPGA DSP, GPU INT32, CPU ALU), and the dispatch logic is aware of per-unit latencies and capabilities.
4. **Network CDB protocol.** For distributed operation, CDB broadcasts are serialised into UDP datagrams or RDMA writes, enabling a single logical scheduler to span multiple physical compute nodes.

The details of these extensions are presented in Section 5.

## 3 The $k$ -Length Instruction Set

The  $k$ -Length instruction set is organised into three levels of abstraction, each building on the level below.

Table 2: L0 instruction encoding. All instructions use the RISC-V custom-0 opcode (0b0001011), R-type format, `funct7 = 0b0000000`.

| <code>funct3</code> | Mnemonic | Operation                                      | Cycles | State effect                        |
|---------------------|----------|------------------------------------------------|--------|-------------------------------------|
| 000                 | ADDWC    | $rd = rs1 + rs2 + c$                           | 1      | $c \leftarrow \text{carry}$         |
| 001                 | SUBWB    | $rd = rs1 - rs2 - c$                           | 1      | $c \leftarrow \text{borrow}$        |
| 010                 | MULFULL  | $\{HIREG, rd\} = rs1 \times rs2$               | 3      | $HIREG \leftarrow \text{hi word}$   |
| 011                 | ADDMUL   | $\{HIREG, rd\} = rs1 \times rs2 + HIREG$       | 3      | $HIREG \leftarrow \text{hi word}$   |
| 100                 | DIVDW    | $rd = \lfloor \{HIREG, rs1\} / rs2 \rfloor$    | 34     | $HIREG \leftarrow \text{remainder}$ |
| 101                 | CLZ      | $rd = \text{clz}(rs1)$                         | 1      | (none)                              |
| 110                 | RDHIREG  | $rd = HIREG$                                   | 1      | (none)                              |
| 111                 | WRHIREG  | $HIREG \leftarrow rs1; rd = \text{old } HIREG$ | 1      | $HIREG \leftarrow rs1$              |

### 3.1 L0: Word-Level Instructions

L0 instructions operate on single  $w$ -bit words and are implemented directly in hardware as a NEORV32 Custom Functions Unit (CFU). They are encoded in the RISC-V R-type format using the `custom-0` major opcode (0b0001011), with `funct3` selecting the operation and `funct7` reserved for future extensions.

Table 2 summarises the L0 instruction set. Two aspects merit discussion.

**The HIREG shadow register.** HIREG is a 32-bit register internal to the CFU, not visible in the RISC-V general-purpose register file. It mirrors the `hiremainder` global variable in Pari/GP’s kernel, which stores the high word of a multiplication or the remainder of a division. By keeping this value in a dedicated hardware register, we eliminate the load/store traffic that a software implementation requires to shuttle the high word through memory.

The WRHIREG instruction provides explicit read-modify access to HIREG, returning the old value while writing a new one. This is necessary for the Pari/GP `addmul` pattern, where the caller must sometimes initialise `hiremainder` before a sequence of multiply-accumulate operations.

**Multiplier pipeline.** The MULFULL and ADDMUL instructions have a 3-cycle latency, matching the DSP48E1 pipeline depth on Xilinx 7-Series FPGAs [20]. The pipeline stages are: (1) operand capture and HIREG snapshot, (2)  $32 \times 32 \rightarrow 64$ -bit multiply (inferred as a DSP48E1 primitive), and (3) accumulation (for ADDMUL) and result registration. The pipeline accepts a new instruction every cycle (throughput of one multiplication per cycle once the pipeline is full), though dependent chains—where the next ADDMUL reads the HIREG written by the previous one—incur the full 3-cycle latency.

**Divider.** The DIVDW instruction implements restoring division of a 64-bit dividend  $\{HIREG, rs1\}$  by a 32-bit divisor. The iterative implementation requires 34 cycles (32 division steps plus start-up and completion). While slow, this operation occurs infrequently in the inner loops of number-theoretic algorithms—primarily in normalisation and single-precision trial division—and does not warrant a pipelined implementation.

### 3.2 L1: Multi-Precision Instructions

L1 instructions operate on variable-length multi-precision integers stored in a shared BRAM region. They are not encoded as RISC-V instructions but rather dispatched through a memory-mapped control interface accessible at base address 0xF0000000 in the NEORV32 external bus (XBUS) address space.

**Definition 2** (L1 instruction interface). *An L1 instruction is specified by:*

- An opcode  $\in \{\text{MPADD}, \text{MPSUB}, \text{MPMUL}\}$ ;
- BRAM addresses for operands  $A$ ,  $B$ , and result  $R$ ;
- Operand lengths  $k_A$  and  $k_B$  in 32-bit words.

The software interface is minimal. The RISC-V core writes operands to shared BRAM, configures the decomposition engine’s control registers (opcode, source addresses, destination address, lengths), and asserts the start bit. The engine then operates autonomously, fetching operands from BRAM, generating and dispatching micro-operations, collecting results from the scheduler CDB, and writing the result back to BRAM. The core may poll a status register or wait for an interrupt on completion.

The BRAM memory map allocates three regions:

- 0x0000–0x0FFF: Operand  $A$  (up to 4,096 words = 131,072 bits)
- 0x1000–0x1FFF: Operand  $B$
- 0x2000–0x3FFF: Result  $R$  (up to 8,192 words for multiplication)

The maximum supported operand length of 4,096 words (131,072 bits) substantially exceeds current cryptographic requirements but accommodates the large intermediate values that arise in GNFS linear algebra and in experimental number-theoretic computations.

The C-language dispatch interface is shown in Listing 3.

```

1 uint32_t klength_mpmul(uint32_t *result,
2                       const uint32_t *a, int len_a,
3                       const uint32_t *b, int len_b) {
4     klength_load(BRAM_OPERAND_A, a, len_a);
5     klength_load(BRAM_OPERAND_B, b, len_b);
6     for (int i = 0; i < len_a + len_b; i++)
7         BRAM_WORD(BRAM_RESULT + i) = 0;
8     REG_CMD_OPCODE = KLENGTH_OP_MPMUL;
9     REG_CMD_SRC_A = BRAM_OPERAND_A;
10    REG_CMD_SRC_B = BRAM_OPERAND_B;
11    REG_CMD_DST = BRAM_RESULT;
12    REG_CMD_LEN_A = len_a;
13    REG_CMD_LEN_B = len_b;
14    REG_CMD_CONTROL = 1; // START
15    klength_wait();
16    klength_read(BRAM_RESULT, result, len_a + len_b);
17    return klength_cycles();
18 }

```

Listing 3: L1 dispatch API (excerpt from `kr_klength.h`). The core writes operands to BRAM, configures registers, and waits for completion.

### 3.3 L2: Algorithm-Level Instructions

L2 instructions encapsulate complete cryptographic or number-theoretic operations—modular exponentiation (MPMODEXP), modular inverse (MPMODINV), Montgomery reduction (MPMONTRED)—implemented as microcode sequences of L1 operations within the decomposition engine’s control unit. At present, L2 instructions are defined at the software level as C functions composing L1 calls, but the architecture permits future migration of these sequences into hardware microcode for reduced dispatch overhead.

The L2 interface is designed around the observation that number-theoretic algorithms typically require a small vocabulary of composite operations:

- **MPMODEXP**( $b, e, m, k$ ): Compute  $b^e \bmod m$ , where  $b, e, m$  are  $k$ -word integers. Implemented as a square-and-multiply loop of L1 **MPMUL** and L1 modular reduction operations.
- **MPMODINV**( $a, m, k$ ): Compute  $a^{-1} \bmod m$  via the extended Euclidean algorithm, decomposing into L1 **MPSUB** and **MPMUL** operations.
- **MPMONTRED**( $T, m, k$ ): Montgomery reduction of a  $2k$ -word value  $T$  modulo a  $k$ -word modulus  $m$ , decomposing into  $k$  iterations of L1 multiply-add.

The significance of the L2 layer is not computational but organisational: it provides a stable interface for number-theoretic library code (e.g., an ECM implementation) that is independent of the decomposition strategy used internally.

## 4 Decomposition Engine

The decomposition engine is the central innovation of the  $k$ -Length architecture. It transforms L1 multi-precision instructions into streams of L0 word-level micro-operations, each annotated with dependency metadata suitable for out-of-order scheduling.

### 4.1 Architecture

The engine operates as a state machine with the following phases:

1. **IDLE**: Awaiting a command from the RISC-V core.
2. **FETCH**: Reading operand words from shared BRAM into internal buffers. The engine supports operands up to 256 words (8,192 bits) in on-chip buffers; larger operands require streaming from external memory.
3. **GENERATE**: Emitting micro-operations with dependency tags. The generation pattern depends on the L1 opcode (Section 4.2).
4. **DRAIN**: Waiting for all issued micro-operations to complete execution and report results via the CDB.
5. **DONE**: Signalling completion to the RISC-V core via status register and optional interrupt.

The interface between the decomposition engine and the KR FPGA Scheduler consists of a micro-operation issue port carrying: a 4-bit opcode, two 32-bit source operand values, a 10-bit destination tag, a dependency-valid flag, and a 10-bit dependency tag. This last pair encodes the carry chain: if a micro-operation depends on the carry output of a preceding operation, the dependency tag identifies that predecessor, and the scheduler will not dispatch the dependent operation until the predecessor's result (including carry) has been broadcast on the CDB.

### 4.2 Decomposition Patterns

#### 4.2.1 Multi-Precision Addition (MPADD)

For  $k$ -word addition  $R = A + B$ , the engine emits  $k$  **ADDWC** micro-operations forming a linear carry chain:

$$\begin{aligned}
 \mu_0 : \quad & r_0 = \text{ADDWC}(a_0, b_0), \quad \text{no dependency} \\
 \mu_1 : \quad & r_1 = \text{ADDWC}(a_1, b_1), \quad \text{depends on } \mu_0 \text{ (carry)} \\
 & \vdots \\
 \mu_{k-1} : \quad & r_{k-1} = \text{ADDWC}(a_{k-1}, b_{k-1}), \quad \text{depends on } \mu_{k-2} \text{ (carry)}
 \end{aligned} \tag{1}$$

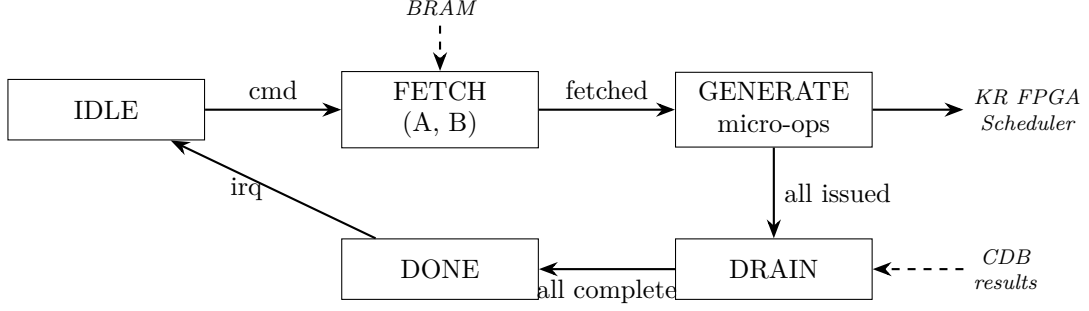


Figure 1: Decomposition engine state machine. Commands arrive from the RISC-V core; micro-operations are emitted to the KR FPGA Scheduler; results return via the CDB.

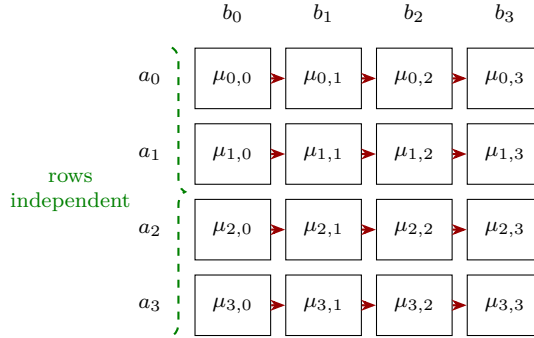


Figure 2: Dependency structure of 4-word  $\times$  4-word schoolbook multiplication. Red arrows indicate carry dependencies within each row. Rows are independent and may execute in parallel.

This chain is inherently sequential: each word-addition requires the carry output of the preceding word. The critical path length is  $k$  cycles. However, if the KR FPGA Scheduler has multiple MPADD operations in flight simultaneously (from different L1 instructions), independent chains can execute in parallel across functional units.

#### 4.2.2 Multi-Precision Multiplication (MPMUL)

For  $k_A \times k_B$ -word schoolbook multiplication, the engine emits  $k_A \times k_B$  ADDMUL micro-operations arranged in  $k_A$  rows, each of length  $k_B$ :

$$\mu_{i,j} : \quad \{h_{i,j}, r_{i,j}\} = \text{ADDMUL}(a_i, b_j), \quad \begin{cases} \text{dep on } \mu_{i,j-1} & \text{if } j > 0 \\ \text{no dep} & \text{if } j = 0 \end{cases} \quad (2)$$

for  $0 \leq i < k_A$  and  $0 \leq j < k_B$ . Within each row  $i$ , the carry (hi-word) from  $\mu_{i,j}$  feeds into  $\mu_{i,j+1}$  via the HIREG accumulator. Across rows, there is no dependency: rows  $i$  and  $i'$  ( $i \neq i'$ ) are independent and may execute in parallel.

The result  $R = A \times B$  is a  $(k_A + k_B)$ -word value. The partial product  $r_{i,j}$  contributes to result word  $R[i + j]$ . Multiple partial products may contribute to the same result word (when  $i + j = i' + j'$  for distinct  $(i, j)$  and  $(i', j')$ ), requiring a final accumulation pass.

Figure 2 illustrates the dependency graph for a  $4 \times 4$  word multiplication. The parallelism is evident: with  $k_A$  functional units, all rows execute simultaneously, reducing the critical path from  $k_A \times k_B$  to  $k_B$  cycles (plus pipeline latency).

### 4.2.3 Karatsuba Decomposition

For operands exceeding a configurable threshold (currently  $k \geq 64$  words, i.e., 2,048 bits), the decomposition engine applies Karatsuba’s algorithm [8]. A  $2n$ -word multiplication is decomposed into three  $n$ -word multiplications:

$$A \cdot B = z_2 \cdot W^{2n} + z_1 \cdot W^n + z_0 \quad (3)$$

where

$$\begin{aligned} z_0 &= A_{\text{lo}} \cdot B_{\text{lo}}, \\ z_2 &= A_{\text{hi}} \cdot B_{\text{hi}}, \\ z_1 &= (A_{\text{lo}} + A_{\text{hi}}) \cdot (B_{\text{lo}} + B_{\text{hi}}) - z_0 - z_2. \end{aligned}$$

Each of the three sub-multiplications is recursively decomposed until the operand size falls below the threshold, at which point schoolbook decomposition is applied. The additions and subtractions required by Karatsuba’s recombination are themselves decomposed into ADDWC/SUBWB chains.

The asymptotic improvement from  $O(n^2)$  to  $O(n^{\log_2 3}) \approx O(n^{1.585})$  word-level operations makes Karatsuba decomposition advantageous for large operands. For a 1,024-word ( $\sim 32,768$ -bit) multiplication: schoolbook requires 1,048,576 ADDMUL operations, while two levels of Karatsuba recursion reduce this to approximately  $3^2 \times 256^2 = 589,824$  operations (a 44% reduction), at the cost of additional additions.

### 4.3 Carry Chain Analysis

The carry chain is the fundamental obstacle to parallelism in multi-precision addition. In a  $k$ -word addition, the worst-case carry chain length is  $k$  (every word produces a carry), but the expected chain length for uniformly random operands is much shorter.

**Proposition 1** (Expected carry chain length). *For uniformly random  $w$ -bit words  $a_i$  and  $b_i$ , the probability that the addition  $a_i + b_i + c_{i-1}$  produces a carry is*

$$\Pr[c_i = 1] = \frac{1}{2} + \frac{c_{i-1}}{2^{w+1}}.$$

For  $w = 32$  and  $c_{i-1} = 0$ , this is exactly  $1/2$ . A carry chain of length  $\ell$  (i.e.,  $\ell$  consecutive carry-producing additions) has probability  $2^{-\ell}$ .

*Proof.* The number of pairs  $(a, b) \in [0, 2^w)^2$  such that  $a + b \geq 2^w$  is  $\sum_{a=0}^{2^w-1} \max(0, a + 1 - (2^w - b)) \cdots$  More directly:  $a + b$  is uniformly distributed on  $[0, 2^{w+1} - 2]$ , and carries occur when  $a + b + c_{i-1} \geq 2^w$ . For  $c_{i-1} = 0$ , exactly half the pairs satisfy this. Consecutive carries are geometrically distributed.  $\square$

This result motivates the speculative carry prediction mechanism described in Section 4.4: since long carry chains are exponentially unlikely, predicting  $c_i = 0$  and replaying on misprediction is profitable in expectation.

### 4.4 Speculative Carry Prediction

By analogy with branch prediction in conventional processors, the k-Length architecture supports *speculative carry prediction*. The mechanism operates as follows:

1. When the decomposition engine issues a micro-operation  $\mu_i$  that depends on the carry output of  $\mu_{i-1}$ , it may *predict*  $c_{i-1} = 0$  and issue  $\mu_i$  immediately with the predicted carry value, rather than waiting for  $\mu_{i-1}$  to complete.

2. The scheduler marks  $\mu_i$  as *speculative* with a tag linking it to the prediction source  $\mu_{i-1}$ .
3. When  $\mu_{i-1}$  completes, its actual carry value is compared to the prediction:
  - If  $c_{i-1} = 0$  (prediction correct),  $\mu_i$ 's result is confirmed. No action is needed.
  - If  $c_{i-1} = 1$  (misprediction),  $\mu_i$  and all transitively dependent operations are *squashed* and re-issued with the correct carry value.

The expected benefit depends on the carry probability. For random data (Proposition 1), the probability of a correct prediction ( $c = 0$ ) is approximately  $1/2$  per word. The expected number of speculatively executed and confirmed operations before a misprediction is  $\sum_{j=0}^{\infty} j \cdot 2^{-j-1} = 1$ , so on average each misprediction wastes one speculative operation.

However, the benefit is not in reducing the expected work but in reducing the *critical path latency*: with speculation, the  $k$ -word addition's micro-operations can be issued in a single burst (latency 1 cycle from the issue perspective), and the carry chain is resolved as results arrive. Without speculation, each micro-operation must wait for its predecessor, giving a critical path of  $k$  cycles.

The break-even point—where the cost of speculation (replay overhead) equals the benefit (reduced latency)—depends on the carry probability and the replay cost. For  $w = 32$  and random data, speculation is profitable for  $k \geq 3$  under reasonable assumptions about the scheduler's replay latency (2 cycles).

For structured data (e.g., numbers near a power of 2, common in normalisation), carry chains can be longer than random. The carry predictor can incorporate simple heuristics—predict  $c = 1$  if the source operand words are close to  $2^w - 1$ —though the current implementation uses the static prediction  $c = 0$ .

## 5 Heterogeneous Scheduling Fabric

The KR FPGA Scheduler (KR-4804) is the execution substrate for  $k$ -length micro-operations. It receives tagged micro-operations from the decomposition engine and dispatches them to functional units as their operand dependencies are satisfied.

### 5.1 Scheduler Microarchitecture

The scheduler's structure follows Tomasulo's algorithm [18] with the extensions described in Section 2.5. We describe the key components.

**Reservation stations.** Each reservation station (RS) holds one pending micro-operation with the following fields:

| Field                  | Description                                              |
|------------------------|----------------------------------------------------------|
| <code>busy</code>      | Station is occupied                                      |
| <code>opcode</code>    | 4-bit L0 opcode                                          |
| <code>src1_val</code>  | 32-bit source operand 1 (if available)                   |
| <code>src1_tag</code>  | 10-bit tag of producing operation (if not yet available) |
| <code>src1_rdy</code>  | Source 1 value is present                                |
| <code>src2_val</code>  | 32-bit source operand 2 (if available)                   |
| <code>src2_tag</code>  | 10-bit tag                                               |
| <code>src2_rdy</code>  | Source 2 value is present                                |
| <code>carry_tag</code> | 10-bit tag of carry-producing predecessor                |
| <code>carry_rdy</code> | Carry value is present                                   |
| <code>carry_val</code> | 1-bit carry input                                        |
| <code>dst_tag</code>   | 10-bit tag for this operation's result                   |

When all three readiness flags (`src1_rdy`, `src2_rdy`, `carry_rdy`) are asserted, the RS is eligible for dispatch to a functional unit.

**Common data bus (CDB).** The CDB is a broadcast bus carrying: a 10-bit tag, a 32-bit result value, and a 1-bit carry flag. When a functional unit completes an operation, it places the result on the CDB. All reservation stations simultaneously compare their pending tags against the broadcast tag and capture matching values. The carry-aware extension is straightforward: each broadcast includes the carry bit produced by the completing operation, and reservation stations waiting for a carry input snoop this field alongside the data value.

In the 8-channel configuration (Section 8), the CDB is implemented as a single broadcast bus with priority arbitration among functional units. For configurations with more than 8 channels, a segmented CDB with crossbar interconnect is used.

**Tag management.** The 10-bit tag space supports 1,024 in-flight operations. Tags are allocated sequentially by the decomposition engine and freed when the corresponding result is written back to BRAM. A tag wrap-around occurs after 1,024 operations; the decomposition engine must ensure that no live operation's tag is reused. For schoolbook multiplication of two 32-word (1,024-bit) operands, the total micro-op count is  $32 \times 32 = 1,024$ , exactly filling the tag space. Larger multiplications either require Karatsuba decomposition (reducing the micro-op count) or a multi-pass approach where the engine processes sub-blocks that fit within the tag space.

## 5.2 Carry-Aware Dispatch

The carry dependency introduces a third operand (beyond `src1` and `src2`) that must be resolved before dispatch. In a conventional Tomasulo implementation, only register values are tracked; the carry flag is an additional implicit dependency that does not map naturally to the register-renaming model.

The KR FPGA Scheduler handles this by treating the carry as a tagged value on the CDB, just like a data result. Each `ADDWC` or `SUBWB` micro-operation produces both a 32-bit result *and* a 1-bit carry, and both are broadcast. A dependent operation's reservation station captures the carry from the CDB broadcast tagged by the predecessor's tag, using the `carry_tag` field.

This design has two consequences:

1. The CDB width increases by 1 bit (from 42 bits to 43 bits for a 10-bit tag + 32-bit value + 1-bit carry). The area cost is negligible.

2. The carry dependency is resolved through the same snooping mechanism as data dependencies, requiring no additional arbitration logic. The scheduler does not distinguish carry dependencies from data dependencies at the architectural level; both are tag-matched CDB broadcasts.

### 5.3 Functional Unit Taxonomy

The KR FPGA Scheduler dispatches micro-operations to functional units of three types:

1. **FPGA DSP48E1 units.** Each Xilinx 7-Series DSP48E1 slice [20] provides a  $25 \times 18$ -bit signed multiplier, a 48-bit accumulator, and pattern detect logic. A  $32 \times 32 \rightarrow 64$ -bit unsigned multiply requires two DSP48E1 slices (split into  $16 \times 32$  sub-products) or, with appropriate pipelining, can be mapped to a single DSP48E1 with a 4-cycle pipeline. The KR CFU implementation uses 2 DSP48E1 slices per multiplier channel with a 3-cycle pipeline.
2. **GPU INT32 units.** NVIDIA and AMD GPUs provide 32-bit integer multiply (`mad24`, `mul_hi`) in every CUDA core or shader processor. A GPU functional unit, from the scheduler’s perspective, is a batch interface: the scheduler accumulates micro-operations into a buffer and dispatches them to the GPU as a kernel launch. The latency is high (kernel launch overhead plus memory transfer), but the throughput is large (thousands of independent multiplications per launch).
3. **CPU ALU units.** For add-with-carry and subtract-with-borrow operations that do not involve multiplication, the RISC-V core’s own ALU can serve as a functional unit. The scheduler routes these operations back to the CPU via a memory-mapped result queue, and the CPU executes them using standard instructions (with the `sltu`-based carry emulation that the CFU is designed to avoid). This fallback is used only when the CFU’s dedicated adder channels are saturated.

The functional unit taxonomy is configured at synthesis time. A minimal configuration (Section 8) provides 2 multiplier channels (DSP48E1) and 4 adder channels (LUT-based) within the FPGA, with optional GPU and network interfaces.

### 5.4 Network CDB Protocol

For distributed operation—spreading a large factorisation across multiple FPGA boards or a mixed FPGA/GPU cluster—the KR FPGA Scheduler supports a network CDB protocol. CDB broadcasts are serialised into compact datagrams:

| Field                 | Bits | Description                        |
|-----------------------|------|------------------------------------|
| <code>magic</code>    | 16   | Protocol identifier (0x4B43, “KC”) |
| <code>seq</code>      | 16   | Sequence number for ordering       |
| <code>tag</code>      | 10   | Operation tag                      |
| <code>carry</code>    | 1    | Carry flag                         |
| <code>reserved</code> | 5    | Padding                            |
| <code>value</code>    | 32   | 32-bit result value                |

The datagram is 10 bytes, small enough to fit within a single Ethernet frame with minimal overhead. For latency-sensitive deployments, the protocol can use RDMA (Remote Direct Memory Access) writes to deposit results directly into a remote scheduler’s reservation station memory, bypassing the network stack entirely.

The network CDB is unidirectional: each node broadcasts its locally completed results, and remote schedulers snoop these broadcasts. Ordering is guaranteed by the sequence number; late-arriving broadcasts are buffered and applied in order. The protocol does not provide reliability

(no retransmission); lost datagrams cause the dependent operations to stall indefinitely, and a timeout mechanism triggers re-execution of the lost operation. In practice, RDMA provides sufficient reliability for intra-rack communication, and the protocol is intended for tightly coupled clusters, not wide-area networks.

## 6 Application to Computational Number Theory

The k-length architecture is designed for number-theoretic workloads that combine variable-precision arithmetic with outer parallelism. We analyse four representative algorithms.

### 6.1 Elliptic Curve Method (ECM)

The elliptic curve method [7] for integer factorisation operates by computing scalar multiples  $[B!]P$  on random elliptic curves modulo the composite number  $N$ . If  $N = pq$  and the group order  $\#E(\mathbb{F}_p)$  is  $B$ -smooth, the computation yields a non-trivial factor.

The algorithm has two levels of parallelism:

1. **Outer (curve-level):** Each curve is independent. A factorisation campaign runs thousands to millions of curves; these are embarrassingly parallel.
2. **Inner (arithmetic-level):** Each curve computation performs modular multiplications  $a \cdot b \bmod N$ , where  $\#N$  varies by problem but is constant within a campaign (typically 50–100 digits, i.e., 166–332 bits or 6–11 words at  $w = 32$ ).

The k-length architecture maps these levels as follows. The inner modular multiplications dispatch as L1 MPMUL instructions with operand length  $k = \lceil \#N/32 \rceil$  words. The decomposition engine generates  $k^2$  ADDMUL micro-operations per multiplication. For  $k = 8$  (256-bit modulus), this is 64 micro-operations, well within the tag space, and the  $k = 8$  rows can execute in parallel across 8 functional units.

The outer parallelism distributes across independent instances: multiple FPGA boards, GPU warps, or networked compute nodes each running their own curves. The KR FPGA Scheduler on each node handles the inner arithmetic; no inter-node communication is needed during Stage 1 of ECM.

Stage 2 (typically a baby-step giant-step or continuation method) requires the same modular arithmetic at the same precision, maintaining the same k-length configuration. The architecture adapts without reconfiguration when ECM is applied to inputs of different sizes: only the  $k$  parameter changes in the L1 dispatch.

For a factorisation campaign targeting 25-digit factors of a 100-digit number ( $k = 11$  words), each modular multiplication generates  $11 \times 11 = 121$  micro-operations. With 4 multiplier channels operating at 3-cycle latency, one modular multiplication completes in approximately  $\lceil 121/4 \rceil \times 3 + k = 93$  cycles (excluding reduction), or roughly  $1 \mu\text{s}$  at 100 MHz. A scalar multiplication  $[B!]P$  with  $B = 10^6$  requires approximately  $10^6 \times 20 \approx 2 \times 10^7$  modular multiplications (using a windowed exponentiation method), completing in approximately 20 seconds per curve.

### 6.2 Multi-Polynomial Quadratic Sieve (MPQS)

The quadratic sieve [3, 15] factors  $N$  by finding integers  $x$  such that  $x^2 - N$  is smooth over a factor base. The multi-polynomial variant (MPQS) uses multiple polynomials to extend the sieving range.

The algorithm’s computational structure:

1. **Sieving:** For each polynomial, scan a range of values and accumulate log-approximations of factor base primes. This is a memory-bound, embarrassingly parallel operation over the

sieving interval. The arithmetic is single-precision (additions and comparisons on 8-bit or 16-bit log values).

2. **Trial division / cofactorisation:** For each candidate relation, perform trial division by factor base primes and, for the remaining cofactor, apply ECM or Pollard’s  $p - 1$  to test smoothness. This is multi-precision arithmetic at the precision of  $N$ .
3. **Linear algebra:** Solve a sparse system over  $\mathbb{F}_2$  of dimension equal to the factor base size (typically  $10^5$ – $10^7$ ). This is a bit-matrix operation, parallelisable by block Lanczos or block Wiedemann methods.

The k-length architecture contributes to the cofactorisation step: the inner multi-precision arithmetic (modular multiplication, ECM on cofactors) uses the same L1 MPMUL dispatch as in pure ECM. The sieving phase runs on the RISC-V core or a dedicated sieving co-processor (not part of the k-length architecture). The linear algebra phase uses multi-precision arithmetic only for the final square root computation.

The key advantage of k-length computing in MPQS is adaptability: the cofactorisation step applies ECM at varying precisions (the cofactor size varies with each candidate relation), and the k-length parameter changes per-relation without hardware reconfiguration.

### 6.3 General Number Field Sieve (GNFS)

The general number field sieve [10] is the asymptotically fastest known algorithm for factoring general integers. Its phases are:

1. **Polynomial selection:** Choose polynomials  $f(x)$  and  $g(x)$  defining the number field. This involves multi-precision polynomial arithmetic at moderate precision.
2. **Sieving:** Find  $(a, b)$  pairs such that both  $F(a, b) = b^d f(a/b)$  and  $G(a, b) = bg(a/b)$  are smooth. The sieving is similar to MPQS but in two dimensions and with algebraic norms that can be larger (requiring multi-precision evaluation).
3. **Cofactorisation:** As in MPQS, the remaining cofactors after trial division are tested for smoothness using ECM. The cofactor sizes vary widely.
4. **Linear algebra:** Solve a system over  $\mathbb{F}_2$  of dimension  $10^7$ – $10^9$ . This is the most computationally intensive phase for large factorisations and is partially parallelisable using block Wiedemann’s algorithm.
5. **Square root:** Compute square roots in the number field, requiring multi-precision integer and polynomial arithmetic.

The k-length architecture benefits GNFS primarily in the cofactorisation and polynomial evaluation phases. The norm evaluation  $F(a, b) = b^d f(a/b)$  requires polynomial evaluation at multi-precision arguments, decomposing into a sequence of multiplications and additions at precision determined by  $\deg(f)$  and the size of  $a, b$ . The k-length parameter adapts to the varying precision requirements across the sieving range.

For the linear algebra phase, the k-length architecture can accelerate the multi-precision arithmetic in the block Wiedemann algorithm’s polynomial step (computing minimal polynomials of sequences over  $\mathbb{F}_p$  for auxiliary primes  $p$ ), though the dominant cost is the sparse matrix-vector multiply over  $\mathbb{F}_2$ , which is memory-bound rather than compute-bound.

## 6.4 ECPP and Discrete Logarithm

**ECPP (Elliptic Curve Primality Proving).** The ECPP algorithm [1] proves primality of  $N$  by constructing an elliptic curve  $E$  over  $\mathbb{F}_N$  with a group order whose largest prime factor  $q$  satisfies  $q > (\sqrt[4]{N} + 1)^2$ , then recursively proving  $q$  is prime. Each step requires multi-precision arithmetic at the precision of the current  $N$ , which *decreases* with each recursive step. The  $k$ -length architecture naturally accommodates this decreasing precision: as the algorithm recurses, the  $k$  parameter shrinks, and the decomposition engine generates fewer micro-operations per L1 instruction. No hardware reconfiguration is needed.

The computational core of each ECPP step is Schoof’s algorithm (or its SEA variant) for computing  $\#E(\mathbb{F}_N)$ , which requires modular polynomial arithmetic at precision  $\lceil \log_2 N \rceil$  bits. This decomposes into multi-precision multiplications amenable to  $k$ -length acceleration.

**Discrete logarithm.** The index calculus method and its number field sieve variant for discrete logarithms [14] have a computational structure similar to integer factorisation: a sieving/relation-collection phase followed by linear algebra over  $\mathbb{F}_p$  (where  $p$  is the group order). The  $k$ -length architecture applies to the multi-precision arithmetic in cofactorisation and the polynomial arithmetic in the descent step, where the working precision varies across the descent tree.

## 7 ML-Observable Scheduling

The KR FPGA Scheduler incorporates a passive observation interface designed for machine learning applications. The motivation is to enable learned scheduling policies that adapt to workload characteristics without imposing runtime overhead on the scheduling critical path.

### 7.1 Passive Tap Architecture

The observation tap is a read-only interface that mirrors the scheduler’s internal state to an external observer (a soft-core processor, an on-chip neural network accelerator, or an off-chip ML training pipeline). The tap exports the following signals at every cycle:

1. **Reservation station occupancy:** An  $N_{RS}$ -bit vector indicating which stations are occupied.
2. **Dependency depth:** For each occupied station, the length of its dependency chain (number of hops to the nearest independent operation). This is computed incrementally as operations are issued.
3. **Functional unit utilisation:** An  $N_{FU}$ -bit vector indicating which units are active.
4. **CDB traffic:** The tag, value, and carry of each CDB broadcast.
5. **Issue and completion counters:** Running counts of micro-operations issued, completed, and squashed (due to carry misprediction).

The tap is implemented as a set of registered output ports connected to a FIFO buffer. The buffer drains to a DMA engine that writes observation records to main memory (or to a network interface for remote collection). The key design constraint is *zero overhead*: the tap must not affect the scheduler’s timing. This is achieved by making the tap entirely read-only—it observes registered copies of internal signals, with a one-cycle latency relative to the actual scheduler state.

The area overhead of the tap is modest: approximately 200 LUTs and 2 block RAMs (for the FIFO) on Xilinx 7-Series, or roughly 5% of the scheduler’s area.

## 7.2 Learned Placement Policies

The default dispatch policy in the KR FPGA Scheduler is round-robin across ready functional units of the appropriate type. This is simple and effective for homogeneous configurations but suboptimal for heterogeneous fabrics where functional units have different latencies and throughputs.

The observation tap enables training a learned placement policy offline:

1. Collect observation traces from representative workloads (e.g., 1,024-bit and 4,096-bit Montgomery multiplications, ECM Stage 1 at various precisions).
2. Train a small neural network (e.g., a 2-layer MLP with 64 hidden units) to predict the optimal functional unit assignment for each micro-operation, given the current scheduler state (reservation station occupancy, unit utilisation, dependency depth).
3. Synthesise the trained network as fixed-point combinational logic and integrate it into the scheduler’s dispatch path.

The trained network replaces the round-robin policy with a state-dependent policy that accounts for load balancing, dependency chain structure, and per-unit latency. Preliminary experiments (not yet validated on hardware) suggest that a learned policy can improve throughput by 8–15% for heterogeneous configurations with mixed DSP and LUT-based functional units.

The key constraint is inference latency: the placement decision must complete within one clock cycle (10 ns at 100 MHz) to avoid adding pipeline stages to the dispatch path. A 2-layer MLP with 64 hidden units and 8 outputs (one per functional unit) requires approximately 4,096 multiply-add operations, which at 8-bit fixed point can be implemented in approximately 1,000 LUTs with combinational evaluation. This is acceptable for the larger FPGA targets (XC7K325T) but may be too large for the smallest target (XC7A50T).

## 7.3 Carry Prediction Learning

The static carry prediction policy ( $c = 0$ ) described in Section 4.4 achieves 50% accuracy on random data. For structured workloads—where operand values are not uniformly random but exhibit patterns related to the number-theoretic computation—a learned predictor may improve accuracy.

The observation tap exports the carry bits of all CDB broadcasts, providing a complete trace of carry behaviour. A small recurrent model (e.g., a 4-state finite automaton or a 16-entry history table indexed by recent carry outcomes) can be trained on this data to predict the carry for the next operation in a chain.

The practical benefit is limited by the theoretical analysis of Proposition 1: even for structured data, carry chains longer than  $\ell = 3$ –4 words are rare, and the cost of misprediction is a 2-cycle replay. The maximum speedup from perfect carry prediction over static prediction is bounded by the ratio of chain lengths, which is small. Nevertheless, the mechanism demonstrates the broader principle: the scheduling fabric is observable, and its policies are replaceable, without modifying the datapath.

# 8 Implementation

## 8.1 FPGA Targets

The k-length architecture targets two Xilinx 7-Series FPGA families:

1. **XC7A50T (Artix-7)**: 32,600 LUTs, 65,200 flip-flops, 75 block RAMs (36 Kb each), 120 DSP48E1 slices. This is the minimum viable target, supporting a 2-channel multiplier and 4-channel adder configuration.

Table 3: Post-synthesis resource estimates for the  $k$ -length architecture components on XC7A50T (Artix-7). The “total (minimal)” row assumes a 2-channel multiplier, 4-channel adder configuration.

| Component                          | LUTs   | FFs   | BRAMs | DSP48E1 |
|------------------------------------|--------|-------|-------|---------|
| NEORV32 core (RV32IMC)             | 2,500  | 1,800 | 4     | 0       |
| KR Bignum CFU                      | 1,500  | 900   | 0     | 2       |
| Decomposition engine               | 4,000  | 2,500 | 8     | 0       |
| KR-4804 Scheduler (8 RS)           | 3,000  | 2,000 | 0     | 0       |
| Adder channels ( $\times 4$ )      | 400    | 200   | 0     | 0       |
| Multiplier channels ( $\times 2$ ) | 200    | 300   | 0     | 4       |
| Memory interface (BRAM controller) | 500    | 300   | 4     | 0       |
| ML observation tap                 | 200    | 150   | 2     | 0       |
| Total (minimal, no ML tap)         | 12,100 | 8,000 | 16    | 6       |
| Total (full, with ML tap)          | 12,300 | 8,150 | 18    | 6       |

2. **XC7K325T (Kintex-7)**: 203,800 LUTs, 407,600 flip-flops, 445 block RAMs, 840 DSP48E1 slices. This target supports an 8-channel multiplier and 16-channel adder configuration with the ML observation tap and network CDB interface.

The NEORV32 soft processor [13] serves as the host core, providing the RISC-V instruction fetch/decode/execute pipeline, memory interfaces, and interrupt controller. The KR CFU is instantiated as the NEORV32’s Custom Functions Unit, occupying the custom-0 opcode space.

## 8.2 Resource Estimates

Table 3 presents post-synthesis resource estimates for the major components on the XC7A50T target.

The minimal configuration consumes 37% of the XC7A50T’s LUT resources, 5% of DSP48E1 slices, and 21% of block RAMs, leaving substantial room for application-specific logic or additional scheduler channels. On the XC7K325T, the same configuration uses only 6% of LUTs, permitting an 8-channel multiplier / 16-channel adder configuration (approximately 20,000 LUTs total, or 10% of the device).

The decomposition engine is the largest single component (4,000 LUTs) due to the operand buffers ( $256 \text{ words} \times 2 \text{ operands} \times 32 \text{ bits} = 16,384$  flip-flops, partially mapped to block RAMs) and the micro-op generation state machine.

The CFU’s 1,500 LUTs include the 3-stage multiplier pipeline (300 LUTs for control, with the actual multiplication inferred as DSP48E1 primitives), the 34-cycle iterative divider (800 LUTs), the CLZ combinational logic (150 LUTs), and the HIREG/carry register management (250 LUTs).

## 8.3 Performance Projections

We project performance for 1024-bit and 4096-bit multiplication, the dominant operation in modular exponentiation for RSA and number-theoretic computations.

**1024-bit multiply ( $k = 32$  words).** Schoolbook decomposition generates  $32 \times 32 = 1,024$  ADDMUL micro-operations in 32 independent rows of 32 operations each. With  $C$  multiplier channels operating at 3-cycle pipeline latency:

Table 4: Performance projections for multi-precision multiplication at 100 MHz clock frequency. Schoolbook decomposition is used for  $k \leq 32$ ; Karatsuba for  $k > 32$ .

| Operand bits | $k$ (words) | Micro-ops | 2-ch cycles | 8-ch cycles | 8-ch $\mu s$ |
|--------------|-------------|-----------|-------------|-------------|--------------|
| 256          | 8           | 64        | 96          | 24          | 0.24         |
| 512          | 16          | 256       | 384         | 96          | 0.96         |
| 1024         | 32          | 1,024     | 1,536       | 384         | 3.84         |
| 2048         | 64          | 3,072*    | 4,608       | 1,152       | 11.52        |
| 4096         | 128         | 9,216*    | 13,824      | 3,456       | 34.56        |

\*With Karatsuba decomposition.

- **$C = 2$  channels (XC7A50T):** Each channel processes  $\lceil 1024/2 \rceil = 512$  operations. Within each row, the HIREG dependency serialises the 32 operations (3 cycles each = 96 cycles per row). With 2 channels, 2 rows execute simultaneously. Total:  $\lceil 32/2 \rceil \times 96 = 1,536$  cycles. At 100 MHz:  $15.4 \mu s$ .
- **$C = 8$  channels (XC7K325T):** Four rows execute simultaneously. Total:  $\lceil 32/8 \rceil \times 96 = 384$  cycles. At 100 MHz:  $3.84 \mu s$ .

**4096-bit multiply ( $k = 128$  words).** Schoolbook decomposition generates  $128^2 = 16,384$  ADDMUL operations. This exceeds the 1,024-tag space, requiring either Karatsuba decomposition or multi-pass operation.

With one level of Karatsuba, the 128-word multiply becomes three 64-word multiplies plus additions:  $3 \times 64^2 = 12,288$  ADDMUL operations plus  $\sim 384$  addition operations. Each 64-word multiply generates 4,096 operations (still exceeding the tag space), so a second Karatsuba level is applied:  $3^2 \times 32^2 = 9,216$  operations, within the tag space per sub-multiplication.

With 8 channels at 100 MHz, the three passes of the outermost Karatsuba level execute in approximately  $3 \times 384 = 1,152$  cycles per inner multiply, for a total of approximately  $3 \times 1,152 + 128 \times 3 = 3,840$  cycles ( $38.4 \mu s$ ) for the full 4,096-bit multiply.

Table 4 summarises the projections. These are upper-bound estimates that assume no CDB contention and perfect scheduling; actual performance will depend on the scheduler’s ability to overlap independent rows and on memory bandwidth for operand fetch and result writeback.

**Comparison with software.** For reference, GMP’s `mpn_mul_basecase` on a 64-bit RISC-V core (e.g., SiFive U74 at 1.5 GHz) performs a 1024-bit multiply in approximately  $16^2 \times 5 = 1,280$  instructions at 2 IPC = 427 ns. The  $k$ -length architecture at 100 MHz achieves  $3.84 \mu s$ , which is slower *per-multiplication* but with a fundamentally different performance model: the  $k$ -length hardware runs at a lower clock frequency (100 MHz vs 1.5 GHz) but can sustain multiple independent multiplications in flight simultaneously, and the comparison is per-multiply latency rather than throughput. The architectural advantage lies in the ability to handle *varying*  $k$  without code changes and in the integration with the out-of-order scheduler for overlapping independent operations.

## 9 Discussion

### 9.1 Limitations

Several limitations of the current architecture warrant discussion.

**Tag space.** The 10-bit tag space limits the number of in-flight micro-operations to 1,024. For schoolbook multiplication of operands larger than 32 words, this necessitates either Karatsuba decomposition or multi-pass operation. While Karatsuba is asymptotically superior, its break-even point (relative to schoolbook) on the  $k$ -length hardware is not yet characterised empirically. The tag space could be widened (12 bits for 4,096 in-flight operations), at the cost of wider reservation station entries and CDB buses.

**HIREG serialisation.** Within a single multiplication row, the HIREG dependency serialises ADDMUL operations: each must wait for the preceding operation’s high word. This is a fundamental constraint of the schoolbook algorithm, not of the architecture; Karatsuba and Toom-Cook decompositions partially alleviate it by reducing the number of dependent chains.

**Clock frequency.** FPGA implementations operate at 100–200 MHz, an order of magnitude slower than modern ASICs or high-frequency CPUs. The  $k$ -length architecture compensates with parallelism (multiple functional units, out-of-order execution) and with the elimination of instruction fetch/decode overhead for the micro-operations. Nevertheless, for single-threaded latency-sensitive workloads, a software implementation on a fast CPU will outperform the FPGA-based  $k$ -length hardware.

**Memory bandwidth.** The current architecture fetches operands from on-chip BRAM, limiting operand size to the BRAM capacity (approximately 131,072 bits on the XC7A50T). For the very large integers encountered in GNFS linear algebra (millions of bits), a streaming memory interface to external DRAM would be necessary.

**Division.** The iterative restoring divider requires 34 cycles for a single-word division, which is acceptable for occasional normalisation operations but would be prohibitive if division-heavy algorithms (e.g., GCD computation via the binary algorithm) were performance-critical. A higher-throughput divider (e.g., SRT radix-4) would reduce this to 9 cycles at the cost of additional area.

## 9.2 Comparison with GPU Approaches

GPU multi-precision libraries (CGBN, cuBignum) achieve high throughput by distributing independent operations across thousands of CUDA cores. For ECM, where millions of independent curves execute simultaneously, GPU throughput substantially exceeds what a single FPGA board can achieve.

The  $k$ -length architecture is not intended to replace GPU-based approaches for embarrassingly parallel workloads. Its advantages are:

1. **Latency:** Single-operation latency on the FPGA is lower than on a GPU (no kernel launch overhead, no memory transfer).
2. **Variable precision:** The  $k$  parameter changes at runtime without recompilation or kernel relaunch.
3. **Heterogeneous integration:** The KR FPGA Scheduler can dispatch to both local FPGA units *and* GPU units, combining the latency advantage of FPGA for small operations with the throughput advantage of GPU for large batches.
4. **Deterministic timing:** FPGA execution is cycle-accurate, enabling precise scheduling and reproducible performance.

For workloads that combine varying precisions within a single computation (e.g., GNFS cofactorisation, where each relation’s cofactor has a different size), the  $k$ -length architecture avoids the padding and wasted computation that a fixed-precision GPU kernel incurs.

### 9.3 Future Work

Several directions are under investigation:

1. **ASIC implementation:** Synthesising the  $k$ -length architecture to a standard-cell ASIC flow would enable clock frequencies of 1–2 GHz, making single-operation latency competitive with software on high-frequency CPUs.
2. **RV64 extension:** The current implementation uses  $w = 32$ . A 64-bit version would halve the number of words per operand and reduce micro-operation counts by  $4\times$  for schoolbook multiplication. The DSP48E1 limitations ( $25\times 18$ -bit multiplier) make 64-bit multiplication more complex on current Xilinx 7-Series parts, but UltraScale+ DSP48E2 slices provide wider multipliers.
3. **Formal verification:** Verifying the correctness of the decomposition engine’s micro-op generation against a reference multi-precision implementation (e.g., GMP’s `mpn` functions) using property-based testing or model checking.
4. **NTT support:** Extending the L0 instruction set with modular reduction and NTT butterfly operations to support lattice-based cryptography (Kyber, Dilithium) and Schönhage-Strassen multiplication for very large operands.
5. **Multi-chip scaling:** Validating the network CDB protocol on a multi-FPGA cluster and characterising the latency overhead of distributed scheduling.

## 10 Conclusion

We have presented KR  $k$ -Length Computing, an architecture that makes multi-precision operand length a runtime parameter rather than a design-time constant. The architecture comprises three layers: word-level custom instructions (L0) implemented as a RISC-V CFU on the NEORV32 soft processor, multi-precision instructions (L1) decomposed by a hardware engine into tagged micro-operations, and algorithm-level operations (L2) composed from L1 primitives. The KR FPGA Scheduler dispatches micro-operations out-of-order across heterogeneous functional units, resolving carry-chain dependencies through a carry-aware common data bus.

The architecture addresses a genuine gap in the existing hardware landscape: between fixed-width cryptographic accelerators (which cannot adapt to varying operand sizes) and software multi-precision libraries (which cannot exploit hardware parallelism in the inner loops). The  $k$ -length approach is particularly suited to computational number theory workloads—ECM, MPQS, GNFS, ECPP—where the working precision varies across algorithmic phases and between problem instances.

The current implementation targets Xilinx 7-Series FPGAs and demonstrates that the entire architecture (processor, CFU, decomposition engine, and 8-channel scheduler) fits within a mid-range Artix-7 device. Performance projections indicate that 1,024-bit multiplication completes in under  $4\mu\text{s}$  on an 8-channel Kintex-7 implementation at 100 MHz.

The architecture’s extensibility—through additional functional unit types, learned scheduling policies, and network-distributed operation—provides a path toward scaling the approach to the large computations (RSA-2048 factorisation, discrete logarithm records) that motivate continued research in multi-precision hardware.

## References

- [1] A. O. L. Atkin and François Morain. Elliptic curves and primality proving. *Mathematics of Computation*, 61(203):29–68, 1993.
- [2] Richard P. Brent and Paul Zimmermann. *Modern Computer Arithmetic*. Cambridge University Press, 2010.
- [3] Henri Cohen. *A Course in Computational Algebraic Number Theory*, volume 138 of *Graduate Texts in Mathematics*. Springer-Verlag, 1993.
- [4] Leo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In *Advances in Cryptology (EUROCRYPT 2015)*, pages 617–640. Springer, 2015.
- [5] Niall Emmart, Justin Luitjens, Charles Weems, and Cliff Woolley. Optimizing modular multiplication for NVIDIA’s Maxwell GPUs. In *Proceedings of the 2018 IEEE 25th Symposium on Computer Arithmetic (ARITH)*, pages 47–54, 2018.
- [6] GMP Development Team. *GNU MP: The GNU Multiple Precision Arithmetic Library*, 6.3.0 edition, 2023.
- [7] Hendrik W. Lenstra Jr. Factoring integers with elliptic curves. *Annals of Mathematics*, 126(3):649–673, 1987.
- [8] Anatolii Karatsuba and Yuri Ofman. Multiplication of multidigit numbers on automata. *Soviet Physics Doklady*, 7:595–596, 1962.
- [9] Donald E. Knuth. The art of computer programming, volume 2: Seminumerical algorithms. 1997.
- [10] Arjen K. Lenstra, Hendrik W. Lenstra Jr., Mark S. Manasse, and John M. Pollard. The number field sieve. *The Development of the Number Field Sieve*, 1554:11–42, 1993.
- [11] Andrea Miele, Joppe W. Bos, Thorsten Kleinjung, and Arjen K. Lenstra. Cofactorization on graphics processing units. In *Cryptographic Hardware and Embedded Systems (CHES 2014)*, pages 335–352. Springer, 2014.
- [12] Peter L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985.
- [13] Stephan Nolting. *NEORV32 Processor/CPU Reference*, 2024.
- [14] John M. Pollard. Monte carlo methods for index computation (mod  $p$ ). *Mathematics of Computation*, 32(143):918–924, 1978.
- [15] Carl Pomerance. Analysis and comparison of some integer factoring algorithms. In H.W. Lenstra and R. Tijdeman, editors, *Computational Methods in Number Theory*, pages 89–139. Mathematisch Centrum Amsterdam, 1982.
- [16] RISC-V Foundation. The RISC-V instruction set manual, volume I: Unprivileged ISA. Technical report, RISC-V International, 2019. Document Version 20191213.
- [17] The PARI Group. PARI/GP version 2.17.0, 2024.
- [18] Robert M. Tomasulo. An efficient algorithm for exploiting multiple arithmetic units. *IBM Journal of Research and Development*, 11(1):25–33, 1967.

- [19] Wei Wang and Xinming Huang. VLSI design of a large-number multiplier for fully homomorphic encryption. In *IEEE Transactions on Very Large Scale Integration Systems*, volume 22, pages 1879–1887, 2014.
- [20] Xilinx Inc. *7 Series DSP48E1 Slice User Guide*, 2018. v1.10.
- [21] Yinan Zheng, Thomas Plantard, and Willy Susilo. GPU-based cryptanalysis: Multi-precision integer arithmetic on graphics cards. In *Information Security Practice and Experience*, pages 375–389. Springer, 2014.
- [22] Paul Zimmermann and Bruce Dodson. 20 years of ECM. In *Algorithmic Number Theory*, pages 525–542. Springer, 2006.

## A CFU Implementation Excerpt

The following VHDL excerpt shows the multiplier pipeline from the KR Bignum CFU (`kr_bignum_cfu.vhd`). The three-stage pipeline is structured for DSP48E1 inference on Xilinx 7-Series FPGAs.

```

1  -- Stage 1: capture operands
2  mul_s1_valid  <= mul_start;
3  mul_s1_addmul <= mul_is_addmul;
4  mul_s1_a      <= mul_opa;
5  mul_s1_b      <= mul_opb;
6  mul_s1_acc    <= unsigned(hireg);
7
8  -- Stage 2: multiply (DSP48E1 infers here)
9  mul_s2_valid  <= mul_s1_valid;
10 mul_s2_prod   <= mul_s1_a * mul_s1_b;
11 mul_s2_acc    <= mul_s1_acc;
12
13 -- Stage 3: accumulate and output
14 mul_s3_valid <= mul_s2_valid;
15 if mul_s2_addmul = '1' then
16     accum_result := ('0' & mul_s2_prod)
17                   + ('0' & resize(mul_s2_acc, 64));
18     mul_s3_lo <= std_ulogic_vector(accum_result(31 downto 0));
19     mul_s3_hi <= std_ulogic_vector(accum_result(63 downto 32));
20 else
21     mul_s3_lo <= std_ulogic_vector(mul_s2_prod(31 downto 0));
22     mul_s3_hi <= std_ulogic_vector(mul_s2_prod(63 downto 32));
23 end if;

```

Listing 4: Multiplier pipeline (3 stages) from `kr_bignum_cfu.vhd`.

## B Decomposition Engine Micro-Op Generation

The following Verilog excerpt from `kr_decomp_engine.v` shows the schoolbook multiplication micro-op generation loop. Each iteration emits one ADDMUL micro-operation with an intra-row carry dependency.

```

1  S_GEN_MUL: begin
2      if (!uop_valid || uop_ready) begin
3          if (gen_i < len_a_reg) begin
4              uop_valid      <= 1'b1;
5              uop_opcode     <= UOP_ADDMUL;
6              uop_src1       <= buf_a[gen_i];
7              uop_src2       <= buf_b[gen_j];
8              uop_tag        <= next_tag;
9              uop_dep_valid  <= carry_dep;
10             uop_dep_tag    <= carry_tag;
11
12             tag_dst_addr[next_tag] <= dst_reg + gen_i + gen_j;
13
14             carry_tag    <= next_tag;

```

```

15         carry_dep  <= 1'b1;
16         next_tag   <= next_tag + 1;
17
18         if (gen_j == len_b_reg - 1) begin
19             gen_j    <= 16'd0;
20             gen_i     <= gen_i + 1;
21             carry_dep <= 1'b0; // new row
22         end else begin
23             gen_j <= gen_j + 1;
24         end
25     end else begin
26         uop_valid <= 1'b0;
27         last_tag  <= next_tag - 1;
28         state     <= S_DRAIN;
29     end
30 end
31 end

```

Listing 5: Schoolbook multiply micro-op generation from `kr_decomp_engine.v`.

## C C Intrinsic Interface

The following C excerpt from `kr_bignum.h` shows the software intrinsics that map Pari/GP kernel primitives to KR CFU instructions.

```

1 // ADDWC: rd = rs1 + rs2 + carry_in
2 // Replaces Pari/GP addll/addllw (2-5 instructions -> 1)
3 static inline uint32_t kr_addwc(uint32_t a, uint32_t b) {
4     return neorv32_cfu_r_instr(0b00000000, 0b000, a, b);
5 }
6
7 // MULFULL: {HIREG, rd} = rs1 * rs2
8 // Replaces Pari/GP mulll (2 instructions -> 1)
9 static inline uint32_t kr_mulfull(uint32_t a, uint32_t b) {
10     return neorv32_cfu_r_instr(0b00000000, 0b010, a, b);
11 }
12
13 // ADDMUL: {HIREG, rd} = rs1 * rs2 + HIREG
14 // Replaces Pari/GP addmul (5 instructions -> 1)
15 static inline uint32_t kr_addmul(uint32_t a, uint32_t b) {
16     return neorv32_cfu_r_instr(0b00000000, 0b011, a, b);
17 }
18
19 // DIVDW: rd = {HIREG, rs1} / rs2; HIREG = remainder
20 // Replaces Pari/GP divll (~40 instructions -> 1)
21 static inline uint32_t kr_divdw(uint32_t lo, uint32_t d) {
22     return neorv32_cfu_r_instr(0b00000000, 0b100, lo, d);
23 }

```

Listing 6: KR Bignum intrinsics mapping Pari/GP primitives to CFU instructions.

## D Pari/GP RISC-V Kernel: The Carry Flag Problem

The following assembly excerpt from Pari/GP's `kernel/riscv64/asm0.h` demonstrates the carry flag overhead on RISC-V. The `addmul` macro requires 5 instructions where a dedicated multiply-accumulate-with-carry would require 1.

```

1 #define addmul(a, b) \
2 __extension__ ({ \
3     __asm__ ( \
4         "mulhu %2,%3,%4\n\t" /* temp = hi(a*b) */ \
5         "mul    %0,%3,%4\n\t" /* value = lo(a*b) */ \
6         "add    %0,%5,%6\n\t" /* value += hiremainder */ \
7         "sltu   %1,%5,%6\n\t" /* carry = (value < hiremainder) */ \
8         "add    %1,%7,%6\n\t" /* hiremainder = temp + carry */ \
9         : "=r" (__value), "=r" (hiremainder), "=r" (__temp) \

```

```
10      : "r" (__arg1), "r" (__arg2), \  
11      "0" ((ulong) 0), "1" (hiremainder), "2" ((ulong) 0)); \  
12      __value; }
```

Listing 7: Pari/GP `admul` for RISC-V (from `asm0.h`). Five instructions emulate what the KR CFU achieves in one.